

ΤΕΧΝΗΤΗ ΝΟΗΜΟΣΥΝΗ (Υπεύθυνος: Καθ. Σ. Λυκοθανάσης)

Ακαδημαϊκό Έτος 2023-24

Εργαστηριακή Άσκηση 3

Ημερομηνία Υποβολής (μέσω e-class) μέχρι τις 08-01-2024, στις 23.55.

Το όνομα του αρχείου θα έχει τη μορφή: XX_YY_ZZ.pdf, όπου XX= Επώνυμο, YY=Αρχικό ονόματος και ZZ=AM. Να δώσετε τις απαντήσεις σας, μετά από κάθε ερώτημα.

Κιουρτ-Μεχμεταλή Αχμέτ
Τεχνητή Νοημοσύνη
Εργαστηριακή Άσκηση 2
AM:1084672
up1084672@ac.upatras.gr

Προγραμματισμός σε Prolog [100 μονάδες]

A. [25 μονάδες]

Δίνεται το παρακάτω πρόγραμμα που υλοποιεί το παιχνίδι «τρίλιζα»:

<https://courses.cs.washington.edu/courses/cse341/03sp/slides/PrologEx/tictactoe.pl.txt>

Κατεβάστε το και δοκιμάστε το στον HY σας. Εξηγείστε σύντομα τη λειτουργία του κατηγορήματος *orespond* και τη λειτουργία του προγράμματος.

Απάντηση:

Το “**orespond**” κατηγορημα στο συγκεκριμένο πρόγραμμα Prolog είναι υπεύθυνο για τη δημιουργία της απόκρισης του υπολογιστή στην κίνηση του χρήστη κατά τη διάρκεια ενός παιχνιδιού Tic-Tac-Toe. Καθορίζει την κίνηση του υπολογιστή με βάση την τρέχουσα κατάσταση του ταμπλό (που αναπαρίσταται ως λίστα) και τους προκαθορισμένους κανόνες παιχνιδιού.

Σύμφωνα με το κώδικα το κατηγορημα “orespond” περιλαμβάνει διάφορες κινήσεις. Αυτές οι κινήσεις είναι:

1. Νικηφόρα Κίνηση

Σύμφωνα με το κώδικα, εάν ο υπολογιστής (παίζοντας ως "o") μπορεί να κάνει μια κίνηση που οδηγεί σε άμεση νίκη, επιλέγει αυτήν την κίνηση. Η συνθήκη win(Newboard, o) ελέγχει εάν η κίνηση οδηγεί σε νίκη για το 'o'. Αυτή η συνθήκη ουσιαστικά είναι:

```
orespond(Board, Newboard) :-  
    move(Board, o, Newboard),  
    win(Newboard, o),  
    !.
```

2. Κίνηση Αποκλεισμού της Νίκης του Αντίπαλου

Σε αυτή την κίνηση εάν δεν υπάρχει άμεση νίκη ούτε ανάγκη να μπλοκαριστεί ο αντίπαλος, ο υπολογιστής κάνει μια κανονική κίνηση. Που αυτή η κίνηση περιγράφεται από το κώδικα παρακάτω:

```
orespond(Board,Newboard) :-  
    move(Board, o, Newboard),  
    not(x_can_win_in_one(Newboard)).
```

3. Απλή Κίνηση

Εάν δεν υπάρχει άμεση νίκη ούτε ανάγκη να μπλοκαριστεί ο αντίπαλος, ο υπολογιστής κάνει μια κανονική κίνηση. Που είναι μία κίνηση απλά για να συνεχιστεί το παιχνίδι χωρίς να υπάρχει πιθανότητας νίκης μετά από την συγκεκριμένη κίνηση.

```
orespond(Board,Newboard) :-  
    move(Board, o, Newboard).
```

4. Παιχνίδι Γάτας (Ισοπαλία)

Ονομάζεται η περίπτωση αν έχουμε το κατηγορήμα "orespond" σε ισοπαλία. Ο πίνακας είναι γεμάτος και δεν υπάρχει νικητής, το παιχνίδι κηρύσσεται ισοπαλία ("Cats game"). Η περίπτωση αυτή στο κώδικα περιγράφεται παρακάτω.

```
orespond(Board,Newboard) :-  
    not(member(b,Board)),  
    !,  
    write('Cats game!'), nl,  
    Newboard = Board.
```

Το συνολικό πρόγραμμα είναι μια απλή υλοποίηση ενός παιχνιδιού Tic-Tac-Toe στην Prolog. Περιλαμβάνει κατηγορήματα για τον έλεγχο των συνθηκών νίκης ("win", "rowwin", "colwin", "diagwin"), χειρισμό κινήσεων παίκτη ("move"), εμφάνιση του πίνακα (εμφάνιση), παιχνίδι ("game" και "selfgame") και αλληλεπίδραση με τον χρήστη ("playo" και "playfrom"). Ο χρήστης παίζει ως "x" και ο υπολογιστής παίζει ως "o". Το πρόγραμμα χρησιμοποιεί μια στρατηγική προοπτικής βάθους 1 για τις κινήσεις του υπολογιστή, προσπαθώντας να εμποδίσει τον αντίπαλο να κερδίσει και στοχεύοντας στη νίκη όταν είναι δυνατόν.

Στο τέλος του προγράμματος έχουν οριστεί η μεταβλητές ώστε να μπορεί να απεικονιστεί το παιχνίδι αλλά και να καταγράφονται οι κινήσεις του χρήστη.

Με λεπτομέρειες μπορούμε να το εξηγήσουμε ως εξής:

win(Board, Player): Ελέγχει εάν ένας παίκτης (Player) έχει κερδίσει το παιχνίδι στον δεδομένο πίνακα Tic-Tac-Toe. Είναι ένα κατηγορήμα ανώτατου επιπέδου που συνδυάζει ελέγχους για νίκες σειρών, στηλών και διαγώνιων.

rowwin(Board, Player): Ελέγχει εάν ένας παίκτης (Player) έχει κερδίσει σε οποιαδήποτε σειρά του πίνακα Tic-Tac-Toe.

colwin(Board, Player): Αυτή την φορά ελέγχει αν ο παίκτης έχει κερδίσει σε οποιαδήποτε στήλη του πίνακα.

diagwin(Board, Player): Ο τελευταίος τρόπος που μπορεί να κερδίσει ένας παίκτης είναι η διαγώνια που αυτό το κατηγορήμα ελέγχει αυτό.

move(Board, Player, Newboard): Αντιπροσωπεύει έναν Player που κάνει μια κίνηση στον πίνακα Tic-Tac-Toe. Δημιουργεί έναν νέο πίνακα (Newboard) με την κίνηση του παίκτη.

game(Board, Player): Αναδρομικό κατηγορήμα που προσομοιώνει το παιχνίδι Tic-Tac-Toe. Εναλλάσσεται μεταξύ παικτών (Παίκτης και αντίπαλος) μέχρι να κερδίσει ένας παίκτης. Που στην συνέχεια περιλαμβάνει παράμετροι για περίπτωση νίκης ή συνέχειας του παιχνιδιού.

selfgame: Ξεκινά το παιχνίδι Tic-Tac-Toe όπου ο υπολογιστής παίζει ενάντια στον εαυτό του. Χρησιμοποιεί το κατηγορήμα του παιχνιδιού για να προσομοιώσει το παιχνίδι.

display(Board): Εμφανίζει την τρέχουσα κατάσταση του πίνακα Tic-Tac-Toe στην οθόνη (Board).

playo: Κατηγορήμα για την έναρξη ενός παιχνιδιού Tic-Tac-Toe με τον χρήστη να παίζει το "x". Καλεί το εξηγητικό κατηγορήμα για να δώσει οδηγίες και ξεκινά το παιχνίδι με το playfrom.

playfrom(Board): Κατηγορήμα που διαχειρίζεται τη ροή του παιχνιδιού. Διαβάζει την κίνηση του χρήστη, ενημερώνει τον πίνακα, τον εμφανίζει, αφήνει τον υπολογιστή να απαντήσει και επαναλαμβάνει μέχρι να ολοκληρωθεί το παιχνίδι.

other(x,o),(x,o): Αυτό το κατηγορήμα συμβολίζει με τη σύμβολο θα παίξει ο δεύτερος παίκτης. Ουσιαστικά, ο πρώτος παίκτης παίζει με "x" σαν πρώτη κίνηση, ο δεύτερος παίκτης πρέπει να παίξει με το "o" σαν πρώτη κίνηση. Που αυτή η παράμετρος ουσιαστικά είναι για να καταλαβαίνει ο υπολογιστής ότι μετά το "x" έρχεται το "o" αφού έχει γραφτεί με αυτό τον τρόπο. Που αυτή η παράμετρος είναι για το selfgame όπως έχει αναφερθεί και στα σχόλια.

x_can_win_in_one: Υπολογίζει αν στην επόμενη κίνηση το "x" μπορεί να κερδίσει το παιχνίδι.

game(Newboard,Otherplayer):Εμφανίζει το νέο πίνακα σε περίπτωση που ακόμα παίζεται το παιχνίδι ώστε για να μπορεί να γίνει η νέα κίνηση από το άλλον παίκτη.

playfrom(Board): Στο τέλος του παιχνιδιού εμφανίζει ποιος είναι ο νικητής με κατάλληλο μήνυμα αν υπάρχει νικητή.

xmove(Board, N, Board): Το κατηγορήμα χρησιμοποιείται για να αναπαραστήσει μια παράνομη κίνηση που έγινε από τον χρήστη που παίζει ως "x". Συγκεκριμένα, όταν ένας χρήστης προσπαθεί να κάνει μια μη έγκυρη κίνηση (π.χ. τοποθετώντας το "x" σε μια ήδη κατειλημμένη θέση), αυτό το κατηγορήμα επικαλείται για να χειριστεί την παράνομη κίνηση. Που μπορούμε να δούμε και στο κώδικα ότι περιγράφονται οι έγκυρες κινήσεις.

orespond(Newboard, Newnewboard): Η περίπτωση που συνεχίζεται το παιχνίδι και ενημερώνεται ο πίνακας, με αποτέλεσμα να μπορεί να κάνει ο παίκτης την νέα κίνηση που παίζει εναντίον του υπολογιστή.

other(Player, Otherplayer): Χρησιμοποιείται για να προσδιορίσει ποιος είναι ο άλλος παίκτης ("Otherplayer") με βάση τον τρέχοντα παίκτη ("Player"). Αυτό διασφαλίζει ότι το παιχνίδι εναλλάσσεται μεταξύ των παικτών "x" και "o".

Παράδειγμα selfgame.

```
?- selfgame.
[x,b,b]
[b,b,b]
[b,b,b]

[x,o,b]
[b,b,b]
[b,b,b]

[x,o,x]
[b,b,b]
[b,b,b]

[x,o,x]
[o,b,b]
[b,b,b]

[x,o,x]
[o,x,b]
[b,b,b]

[x,o,x]
[o,x,o]
[b,b,b]

[x,o,x]
[o,x,o]
[x,b,b]

[x,o,x]
[o,x,o]
[x,o,b]

[player,x,wins]
true.
```

Εικόνα 1: Παράδειγμα Υλοποίησης με χρήση Selfgame.

Παράδειγμα playo.

```
?- playo.
| playo.
You play X by entering integer positions followed by a period.
[1,2,3]
[4,5,6]
[7,8,9]

|: 5.
[b,b,b]
[b,x,b]
[b,b,b]

[o,b,b]
[b,x,b]
[b,b,b]

|: 8.
[o,b,b]
[b,x,b]
[b,x,b]

[o,o,b]
[b,x,b]
[b,x,b]

|: 3.
[o,o,x]
[b,x,b]
[b,x,b]

[o,o,x]
[b,x,b]
[o,x,b]

|: 4.
[o,o,x]
[x,x,b]
[o,x,b]

[o,o,x]
[x,x,o]
[o,x,b]

|: 9.
[o,o,x]
[x,x,o]
[o,x,x]

Game game!
[o,o,x]
[x,x,o]
[o,x,x]

|:
```

Εικόνα 2: Παράδειγμα Υλοποίησης με χρήση playo.

B. [25 μονάδες]

Δίνεται το παρακάτω πρόγραμμα που υλοποιεί το παιχνίδι «τρίλιζα»:

<https://wiki.ubc.ca/Course:CPSC312-2021/Tic-Tac-Toe>

Κατεβάστε το και δοκιμάστε το στον ΗΥ σας. Εξηγείστε σύντομα τη βασική διαφορά της λειτουργίας του από το πρόγραμμα του μέρους Α.

Απάντηση:

Η κύρια διαφορά στη λειτουργία μεταξύ του πρώτου και του δεύτερου προγράμματος έγκειται στην προσέγγισή τους στην αναπαράσταση της κατάστασης του παιχνιδιού και στη λογική για τον καθορισμό της επόμενης κίνησης.

Στο πρώτο πρόγραμμα, η κατάσταση του παιχνιδιού αναπαρίσταται ως μια λίστα με εννέα στοιχεία, όπου κάθε στοιχείο αντιστοιχεί σε μια θέση στον πίνακα Tic-Tac-Toe. Τα στοιχεία μπορεί να είναι είτε "X", "O" ή "b" (κενό). Το πρόγραμμα χρησιμοποιεί κατηγορήματα Prolog για να ορίσει τις συνθήκες νίκης, τις κινήσεις του παίκτη και τη συνολική λογική του παιχνιδιού. Ο αντίπαλος υπολογιστής στο πρώτο πρόγραμμα υλοποιείται με μια απλή στρατηγική βασισμένη σε κανόνες.

Στο δεύτερο πρόγραμμα, η κατάσταση του παιχνιδιού αναπαρίσταται ως μια λίστα με εννέα ακέραιους αριθμούς, όπου το 1 αντιπροσωπεύει το "X", το -1 αντιπροσωπεύει το "O" και το 0 αντιπροσωπεύει ένα κενό σημείο. Το δεύτερο πρόγραμμα χρησιμοποιεί τον αλγόριθμο minimax για τον αντίπαλο υπολογιστή, καθιστώντας τον πιο περίπλοκο από τη στρατηγική που βασίζεται σε κανόνες στο πρώτο πρόγραμμα. Ο αλγόριθμος minimax διερευνά συστηματικά πιθανές κινήσεις και τα αποτελέσματά τους για τη λήψη βέλτιστων αποφάσεων.

Η μεγαλύτερη διαφορά βέβαια από το πρώτο πρόγραμμα είναι ότι ο δεύτερο κώδικας δεν έχει την λειτουργία (mode) ο υπολογιστής να παίξει με το εαυτό του. Αλλά ο δεύτερος κώδικας έχει την λειτουργία μόνο να παίξει με αντίπαλο τον χρήστη. Επίσης αυτή την φορά μπορούμε να διαλέξουμε αν θέλουμε να έχουμε τα "X" ή τα "O" σαν εντολή.

Ο αλγόριθμος minimax είναι ένας αναδρομικός αλγόριθμος που χρησιμοποιείται για τη λήψη αποφάσεων σε παιχνίδια δύο παικτών με τέλειες πληροφορίες, όπως το tic-tac-toe. Εξερευνά όλες τις πιθανές καταστάσεις του παιχνιδιού, εκχωρώντας μια τιμή σε κάθε κατάσταση με βάση το αποτέλεσμα του παιχνιδιού (νίκη, ήττα ή ισοπαλία). Ο αλγόριθμος υποθέτει ότι και οι δύο παίκτες παίζουν βέλτιστα.

Περιέχει 9 σημεία για κίνηση το παιχνίδι.

Κατηγορήματα

- Το **"flip"** ανταλλάσσει παίκτες. Παίρνει την αναπαράσταση ενός παίκτη (1 ή -1) και επιστρέφει την αναπαράσταση του άλλου παίκτη.
- Το **"win_position"** ελέγχει εάν ένας δεδομένος παίκτης έχει κερδίσει το παιχνίδι. Επιστρέφει "true" εάν ο παίκτης έχει τρία στη σειρά οριζόντια, κάθετα ή διαγώνια.
- Το **"game_end"** ελέγχει εάν το παιχνίδι έχει τελειώσει. Επιστρέφει το αποτέλεσμα για έναν συγκεκριμένο παίκτη (νίκη, ήττα ή ισοπαλία).

- Το **“move”** ενημερώνει την κατάσταση του παιχνιδιού τοποθετώντας ένα πλακίδιο στον πίνακα για έναν συγκεκριμένο παίκτη. Επιστρέφει τη νέα κατάσταση πλακέτας.
- Το **“possible_moves”** δημιουργεί μια λίστα με πιθανές κινήσεις (δείκτες κενών σημείων) στον τρέχοντα πίνακα.
- Το κύριο σημείο εισόδου είναι το κατηγορημα **“minimax_player”** το οποίο παίρνει τον τρέχοντα πίνακα, παίκτη και επιστρέφει την καλύτερη κίνηση για αυτόν τον παίκτη.
- Το **“minimax”** υπολογίζει την ελάχιστη τιμή για κάθε πιθανή κίνηση και το **“argmax”** επιλέγει την κίνηση με την υψηλότερη τιμή για τον τρέχοντα παίκτη.
- Το **“move_value”** υπολογίζει την τιμή μιας κίνησης με βάση την κατάσταση του πίνακα που προκύπτει χρησιμοποιώντας τον αλγόριθμο minimax.
- Το **“board_value”** υπολογίζει την τιμή της τρέχουσας κατάστασης του πίνακα. Εάν το παιχνίδι τελειώσει, επιστρέφει το αποτέλεσμα. Διαφορετικά, καλεί τον αλγόριθμο minimax για τον επόμενο παίκτη.

Ο αλγόριθμος minimax υλοποιείται χρησιμοποιώντας αναδρομή, εξερευνώντας όλες τις πιθανές κινήσεις και τα αποτελέσματά τους. Υπάρχουν λειτουργίες για την εμφάνιση της τρέχουσας πλακέτας και την προτροπή του χρήστη για τη μετακίνησή του.

- Το **“play”** χειρίζεται τη ροή του παιχνιδιού, εναλλάσσοντας μεταξύ των παικτών ανθρώπου και υπολογιστή.
- Το τελευταίο κατηγορημα **“game_over”** εμφανίζει το αποτέλεσμα του παιχνιδιού και ζητά από τον χρήστη εάν θέλει να παίξει ξανά.
- Με την εκτέλεση της εντολής **“start”** θα ζητηθεί να επιλεγεί το Χ ή Ο. Στη συνέχεια, το παιχνίδι θα συνεχίσει με τον επιλεγμένο παίκτη να κάνει κινήσεις εναλλάξ μέχρι να τελειώσει το παιχνίδι.

Συνοπτικά, το δεύτερο πρόγραμμα χρησιμοποιεί μια διαφορετική αναπαράσταση για την κατάσταση του παιχνιδιού και εφαρμόζει μια πιο προηγμένη στρατηγική για τον αντίπαλο υπολογιστή μέσω του αλγόριθμου minimax, παρέχοντας μια πιο έξυπνη και στρατηγική εμπειρία παιχνιδιού.

Παράδειγμα Υλοποίησης:

```
?- start.  
Select [0] to choose X, or [1] to choose O. End your choice with a period: 0.  
  
  |  |  
-----  
  |  |  
-----  
  |  |  
Select an integer from 0 to 8 inclusive. End your choice with a period |: 4.  
The computer made a move  
  
  O |  |  
-----  
  | X |  
-----  
  |  |  
Select an integer from 0 to 8 inclusive. End your choice with a period |: 1.  
The computer made a move  
  
  O | X |  
-----  
  | X |  
-----  
  | O |  
Select an integer from 0 to 8 inclusive. End your choice with a period |: 6.  
The computer made a move  
  
  O | X | O  
-----  
  | X |  
-----  
  X | O |  
Select an integer from 0 to 8 inclusive. End your choice with a period |: 3.  
The computer made a move  
  
  O | X | O  
-----  
  X | X | O  
-----  
  X | O |  
Select an integer from 0 to 8 inclusive. End your choice with a period |: 8.  
  
  O | X | O  
-----  
  X | X | O  
-----  
  X | O | X  
Tie!  
  
Do you want to play again?  
Select [1] for Yes or [0] for No |: █
```

Εικόνα 3: Υλοποίηση δεύτερου κώδικα

Όταν δοκιμάσουμε το δεύτερο πρόγραμμα όπως δοκιμάσαμε και το πρώτο παρατηρούμε ότι αυτή την φορά υπάρχει και η ερώτηση για το αν θέλουμε να ξαναπαίξουμε το παιχνίδι:

Γ. [50 μονάδες]

Σκεφτείτε ένα τρόπο για να επιδείξετε την λειτουργία του προγράμματος A εναντίον του προγράμματος B. Σχολιάστε σύντομα τη σχετική τους επίδοση και να εξηγήσετε ποιος παίκτης και με ποια στρατηγική μπορεί να κερδίζει;

Σημείωση: Για να είναι πλήρης η απάντηση, θα πρέπει να έχετε μερικά στιγμιότυπα-screen shots (από το περιβάλλον της Prolog) στα οποία να επιδεικνύετε δύο (2) διαφορετικά παιχνίδια.

Απάντηση:

Σύμφωνα με τους 2 κωδικούς από τις παραπάνω ερωτήσεις. Παρατηρούμε ότι ο πρώτος χρησιμοποιεί μία κατανόηση που αρχικά βλέπει αν υπάρχει η περίπτωση να νικήσει ο χρήστης με την επόμενη του κίνηση, έτσι ώστε να μπλοκάρει τον χρήστη για να μην κερδίσει. Αν δεν υπάρχει η περίπτωση νίκης του χρήστη στο επόμενο βήμα απλά κάνει μία κίνηση χωρίς να υπάρχει κάποια φιλοσοφία από πίσω. Ουσιαστικά ο πρώτος κώδικας έχει την νοημοσύνη να προσπαθεί να μην κερδίσει ο άλλος παίκτης.

Ο δεύτερος κώδικας, όπως έχουμε αναφερθεί και παραπάνω χρησιμοποιεί το αλγόριθμο minimax. Ο αλγόριθμος minimax στο tic-tac-toe λειτουργεί εξερευνώντας αναδρομικά όλες τις πιθανές κινήσεις, εκχωρώντας το σκορ με βάση το αποτέλεσμα του παιχνιδιού (νίκη, ήττα ή ισοπαλία) και εναλλάσσοντας τη μεγιστοποίηση και την ελαχιστοποίηση των παικτών. Υποχωρεί για να καθορίσει την καλύτερη κίνηση για κάθε παίκτη, με στόχο να μεγιστοποιήσει το σκορ για τον τρέχοντα παίκτη και να το ελαχιστοποιήσει για τον αντίπαλο. Ο αλγόριθμος συνεχίζει μέχρι να φτάσει σε μια τερματική κατάσταση και οι επιλεγμένες κινήσεις οδηγούν σε βέλτιστα αποτελέσματα με τέλειο παιχνίδι. Ουσιαστικά, δεν προσπαθεί να μπλοκάρει μόνο το παίκτη αλλά και να κάνει μία κίνηση που θα είναι χρήσιμη και για την συνέχεια του παιχνιδιού για το εαυτό του.

Καταλήγουμε ότι ο δεύτερος κώδικας έχει μεγαλύτερη πιθανότητα να κερδίσει τον πρώτο κώδικα λόγω της στρατηγικής minimax που παρέχει αφού σκέφτεται και το μέλλον.

Ας εξηγήσουμε αυτές τις υλοποιήσεις και με παράδειγμα από κάθε κώδικα:

Παράδειγμα 1^{ου} κώδικα

```
I      playo.
You play X by entering integer positions followed by a period.
[1,2,3]
[4,5,6]
[7,8,9]

I: 1.
[x,b,b]
[b,b,b]
[b,b,b]

[x,o,b]
[b,b,b]
[b,b,b]

I: ■
```

Εικόνα 4: Επίδειξη μεθοδολογίας 1^{ου} κώδικα

Αν πάρουμε την μία γωνία βλέπουμε ότι το πρόγραμμα αντιδρά σε τύχη αφού δεν σκέφτεται να κάνει την κίνηση ώστε να πάρει το κέντρο που είναι μία γνωστή κίνηση και για διευκόλυνση στην συνέχεια του παιχνιδιού.

Παράδειγμα 2^{ου} κώδικα

```
?-
|
|      start.
|
Select [0] to choose X, or [1] to choose O. End your choice with a period: 0.

  |  |
  ---
  |  |
  ---
  |  |
Select an integer from 0 to 8 inclusive. End your choice with a period |: 0.
The computer made a move

X |  |
  ---
  | O |
  ---
  |  |
Select an integer from 0 to 8 inclusive. End your choice with a period |:
```

Εικόνα 5: Επίδειξη μεθοδολογίας 2^{ου} κώδικα

Βλέπουμε ότι ο δεύτερος κώδικας μετά την κίνηση που κάνουμε στην γωνία επιλέγει να κάνει την πρώτη του κίνηση στο κέντρο του παιχνιδιού για να μπορεί να καλύψει περισσότερες πιθανότητες και στο μέλλον.

Στην συνέχεια στο πρώτο παράδειγμα παρατηρούμε ότι έχουμε:

```
?- selfgame.
[x,b,b]
[b,b,b]
[b,b,b]

[x,o,b]
[b,b,b]
[b,b,b]

[x,o,x]
[b,b,b]
[b,b,b]

[x,o,x]
[o,b,b]
[b,b,b]

[x,o,x]
[o,x,b]
[b,b,b]

[x,o,x]
[o,x,o]
[b,b,b]

[x,o,x]
[o,x,o]
[x,b,b]

[x,o,x]
[o,x,o]
[x,o,b]

[player,x,wins]
true.
```

Εικόνα 6: Παρατήρηση χρήσης κώδικα με εντολή selfgame.

Αν κοιτάξουμε στο τρίτο βήμα που έχει και την σειρά ο παίκτης “X” δεν προσπαθεί να νικήσει το παιχνίδι αλλά μόνο να κάνει μία κίνηση αφού δεν υπάρχει ο κίνδυνος να κερδίσεις ο άλλος παίκτης. Που σημαίνει ότι είναι μία άχρηστη κίνηση.

Ενώ αν ξανακοιτάξουμε το δεύτερο παράδειγμα:

```
?- start.
Select [0] to choose X, or [1] to choose O. End your choice with a period: 0.

  |  |
  ---
  |  |
  ---
  |  |
Select an integer from 0 to 8 inclusive. End your choice with a period |: 3.
The computer made a move

  O |  |
  ---
  X |  |
  ---
  |  |
Select an integer from 0 to 8 inclusive. End your choice with a period |: 8.
The computer made a move

  O |  | O
  ---
  X |  |
  ---
  |  | X
Select an integer from 0 to 8 inclusive. End your choice with a period |: 6.
The computer made a move

  O | O | O
  ---
  X |  |
  ---
  X |  | X
O won!

Do you want to play again?
Select [1] for Yes or [0] for No |: 
```

Εικόνα 7: Ανάλυση βημάτων 2^{ου} κώδικα

Αν κάνουμε σαν χρήστης μία άσκοπη κίνηση γράφοντας 8. σαν επόμενη εντολή βλέπουμε ότι ο αλγόριθμος δεν κάνει μία άχρηστη κίνηση όπως το πρώτο παράδειγμα, αλλά κάνει ένα βήμα που θα το οδηγήσει σε νίκη έχοντας το πλεονέκτημα ο υπολογιστής εναντίον του χρήστη λόγο της άχρηστης κίνησης του χρήστη.

Αν κάνουμε το ίδιο παράδειγμα με την εντολή playo. και στο πρώτο παράδειγμα βλέπουμε ότι θα έχουμε τις ίδιες κινήσεις όπως και στο δεύτερο παράδειγμα.

```

?- playo.
You play X by entering integer positions followed by a period.
[1,2,3]
[4,5,6]
[7,8,9]

I: 4.
[b,b,b]
[x,b,b]
[b,b,b]

[o,b,b]
[x,b,b]
[b,b,b]

I: 9.
[o,b,b]
[x,b,b]
[b,b,x]

[o,o,b]
[x,b,b]
[b,b,x]

I: 7.
[o,o,b]
[x,b,b]
[x,b,x]

[o,o,o]
[x,b,b]
[x,b,x]

I win!
true .

```

Εικόνα 8: Ανάλυση βημάτων 1^{ου} κώδικα

Αλλά με βάση της εικόνας 4 και 5 που έχουμε αναλύσει βλέπουμε ότι ο δεύτερος έχει πάντα την καλύτερη πιθανότητα νίκης λόγω της απόκρισης που περιέχει ο κώδικας του. Ο αλγόριθμος δουλεύει με μία μεθοδολογία ενώ ο πρώτος κώδικας έχει μία προσέγγιση ποιο μη ασφαλές που μπορούμε να το παρατηρήσουμε αυτό και από τις εικόνες.